Appendix C: IEEE 830

Software Requirements Specification

for

«Habit Tracker website»

Version 1.0

Prepared by Yelbassy Shugyla

Lodz University of Technology

11.03.26

Table of Contents

1. Introduction ……. 3

1.1.Purpose …… 3

1.2. Document Conventions ……...……3

1.3. Intended Audience and Reading Suggestions …….3

1.4. Product Scope …….3

2. Overall Description …… 4

2.1. Product Perspective ……4

2.2. Product Features …….4

2.3. User Classes and Characteristics ……4

2.4. Operating Environment ……..4

2.5. Design and Implementation Constraints ……4

2.6. User Documentation ……...4

2.7. Assumptions and Dependencies ……..4

3. System Features ……5

3.1. User Account Management …….5

3.2. Habit Management Interface ……5

3.3. Progress Dashboard ……5

4. External Interface Requirements ……5

4.1. User Interfaces Overview ……5

4.2. Hardware Interfaces ……5

4.3. Software Interfaces ……..5

5. System Features/Modules ……6

5.1. User Account Management ……6

5.1.1 Description and Priority ……..6

5.1.2 Stimulus/Response Sequences ……...6

5.1.3 Functional Requirements ……6

5.2. Habit Management Interface ……6

5.2.1 Description and Priority ……6

5.2.2 Stimulus/Response Sequences ……6

5.2.3 Functional Requirements ……6

5.3. Progress Dashboard ……7

5.3.1 Description and Priority ……7

5.3.2 Stimulus/Response Sequences ……7

5.3.3 Functional Requirements ……7

6. Nonfunctional Requirements ……7

6.1. Performance ……7

6.2. Security ……7

List of Figures ……8

Figure 1. Use Case Diagram for the Habit Tracker System ……8

Figure 2. Class Diagram of the Application Architecture ……8

Figure 3. State Diagram ……8

Figure 4. Activity Diagram ...……8

Figure 5. Sequence Diagram ……8

Revision History

Name	Date	Reason for Changes	Version
Yelbassy Shugyla	13.03.26	Initial version	1.0
Yelbassy Shugyla	16.03.26	Removed idea with visual moodboard	1.1
Yelbassy Shugyla	21.03.26	Added UML Use case diagram	1.2
Yelbassy Shugyla	28.03.26	Added Class diagram	1.3
Yelbassy Shugyla	10.04.26	Added UML Sequence / State / Activity Diagram	1.4
Yelbassy Shugyla	16.04.26	Editing / correcting mistakes	1.5

1. Introduction

1.1. Purpose

This document specifies the software requirements and design constraints for the "Habit Tracker" web application, specifically version 1.0. It covers the full scope of the frontend interface and backend logic required to manage user habits and motivational data.

1.2. Document Conventions

This specification follows the IEEE 830 standard and the SMART criteria for requirement engineering. The keyword "**shall**" is used to represent a mandatory engineering obligation, while all priorities for high-level requirements are inherited by their detailed sub-requirements.

1.3. Intended Audience and Reading Suggestions

The primary audience includes university developers and project managers who will implement the system using FastAPI. It also serves as a guide for testers to verify that each functional requirement meets the specified performance and security standards

1.4. Product Scope

The Habit Tracker is a productivity tool designed to help users build consistency through a digital habit list and a visual moodboard. Its primary goal is to provide a structured environment where students can monitor their daily routines and stay motivated through persistent data tracking.

2. Overall Description

2.1. Product Perspective

This product is a new, self-contained web application developed as part of a Software Engineering laboratory task. It serves as a modern replacement for manual paper-based tracking, utilizing web technologies to provide real-time updates.

2.2. Product Features

The system allows users to create, edit, and delete personal habits through a student-style HTML interface. The application includes a secure login system to protect individual user data and progress history.

2.3. User Classes and Characteristics

The primary user class is the Individual End-User. These users are individuals seeking to track personal habits. They are expected to have basic web browsing skills. Their frequency of use is expected to be daily. All users have access to the same core features on the website.

2.4. Operating Environment

The website will operate in a standard client-server environment:

Client-side: Any modern web browser (Chrome, Firefox, Safari, Edge) on desktop, tablet, or mobile devices.

Server-side: A web server (exact technology to be determined later) accessible via the internet.

2.5. Design and Implementation Constraints

The website shall be developed as a client-server web application with a responsive user interface that functions correctly on desktop, tablet, and mobile screen sizes, following consistent web design conventions for ease of use, and ensuring that all user data shall be private and accessible only to the authenticated user.

2.6. User Documentation

The website will include a "Help" section with FAQs and a quick start guide. Tooltips may be used throughout the interface to guide new users.

2.7. Assumptions and Dependencies

It is assumed that users will have a stable internet connection to access the website and basic knowledge of using a web browser; furthermore, the project depends on the availability of a reliable web hosting environment for deployment.

3. System Features

This section provides a brief introduction to the website's main features.

3.1. User Account Management

This feature enables users to register for a new account, log in to existing accounts, and securely log out. It ensures that each user's personal habit data is protected and accessible only to them.

3.2. Habit Management Interface

This feature allows users to create, view, edit, and delete the habits they wish to track. Users can organize their personal development goals by defining habit names and managing their list of tracked activities.

3.3. Progress Dashboard

This feature provides users with a daily overview of their habits and allows them to mark completion with a single click. It also displays current streaks and historical progress to motivate users and show their consistency over time.

4. External Interface Requirements

4.1. User Interfaces Overview

High-Level Functionality: After logging in via a simple email/password form, the user lands on the Main Dashboard, which displays today's habits with checkboxes. The user clicks a checkbox to mark a habit complete, and the system immediately updates the display. From a consistent navigation menu, the user can access the Habit Management Page to add, edit, or delete habits by typing names into text fields and clicking buttons. The Progress Page shows historical data in a calendar view or charts, where users can click on past dates to edit entries. All interactions use standard web controls.

Minimum Requirements: The interface shall maintain consistent design with uniform colors, fonts, and button styles across all pages, feature a responsive layout that adapts to desktop, tablet, and mobile screens, include standard elements such as navigation menu, Logout button, and Help link on every page, support basic keyboard navigation, and display error messages in red text near the relevant field explaining how to fix the issue.

4.2. Hardware Interfaces

There are no direct hardware interface requirements. The website will interact with standard input devices (keyboard, mouse, touchscreen) through the user's web browser and operating system.

4.3. Software Interfaces

The website will interface with the following software components:
Web Browsers: The client-side code (HTML, CSS, JavaScript) shall be compatible with the latest two major versions of Chrome, Firefox, Safari, and Edge.
Backend Server: The frontend (website in the browser) shall communicate with a backend server via a RESTful API (or similar) to retrieve and store user data.

Email Service: The system may interface with an email service provider to send reminder emails to users.
Browser Notification API: The system may use the browser's built-in Notification API to send push reminders if the user grants permission.

5. System Features/Modules

5.1. User Account Management

5.1.1 Description and Priority

This feature allows users to create a personal account and securely access the website. Priority: High.

5.1.2 Stimulus/Response Sequences

Stimulus: User navigates to the registration page and submits an email and password.

Response: The system shall create a new user account and automatically log the user into the website.

Stimulus: An existing user submits their email and password on the login page.

Response: The system shall authenticate the credentials and redirect the user to their private dashboard.

Stimulus: An authenticated user clicks the "Logout" button in the navigation menu.

Response: The system shall end the user's session and redirect them to the public login page.

5.1.3 Functional Requirements

The website shall provide a registration form for a new user to create an account using a unique email address and a password, and if the email address is already registered, the system shall display an error message indicating that the email is already in use; the website shall provide a login form for a registered user to access their account using their email and password, and shall display an appropriate error message if a user enters incorrect login credentials, using a generic message such as "Invalid email or password" when the email is not found or the password does not match; additionally, the website shall provide a "Logout" button for an authenticated user to end their session.

FR-01: The website shall provide a registration form for a new user to create an account using a unique email address and a password
FR-02: The website shall provide a login form for a registered user to access their account using their email and password
FR-03: The website shall provide a "Logout" button for an authenticated user to end their session
FR-04: The website shall display an appropriate error message if a user enters incorrect login credentials

5.2. Habit Management Interface

5.2.1 Description and Priority

This feature allows users to define and manage the habits they want to track through the web interface. Priority: High.

5.2.2 Stimulus/Response Sequences

Stimulus: An authenticated user clicks an "Add New Habit" button and enters a habit name in a form.

Response: The system shall save the new habit and display it in the user's list of habits.

Stimulus: An authenticated user clicks an "Edit" icon next to a habit and changes its name.

Response: The system shall update the habit's name and display the updated list.

Stimulus: An authenticated user clicks a "Delete" icon next to a habit and confirms the deletion.

Response: The system shall permanently remove the habit and all its associated log entries from the user's account.

5.2.3 Functional Requirements

FR-05:The website shall provide a form for an authenticated user to create a new habit by specifying a name (e.g., "Read for 30 minutes").
FR-06: The website shall display a list of all active habits for the authenticated user on the Habit Management page.
FR-07: The website shall allow an authenticated user to edit the name of an existing habit.
FR-08: The website shall allow an authenticated user to delete an existing habit after a confirmation prompt

5.3. Progress Dashboard

5.3.1 Description and Priority

This core feature allows users to log their daily progress and view their consistency over time directly on the website. Priority: **High**.

5.3.2 Stimulus/Response Sequences

Stimulus: On the main dashboard, a user clicks the checkbox next to a habit for the current day.

Response: The system shall record that the habit was completed on today's date and immediately update the display (e.g., show a checkmark).
Stimulus: A user clicks on a habit name to view its detailed progress.

Response: The website shall navigate to a detail page showing the habit's completion history (e.g., a calendar view) and current streak count.
Stimulus: A user clicks on a past date in the calendar view and marks a habit as complete.

Response: The system shall log the completion for that past date and recalculate the streak.

5.3.3 Functional Requirements

FR-09: The main dashboard shall display all of the user's active habits for the current day, each with an interactive element (e.g., checkbox) to mark completion.
FR-10: The system shall allow a user to mark a habit as "completed" for the current day with a single click.
FR-11: The system shall calculate and display the user's current streak (number of consecutive days completed) for each habit on the dashboard or detail page.
FR-12: The website shall provide a view (e.g., a calendar or list) that shows the user's habit completion history for past days.

FR-13: The system shall allow a user to mark a habit as "completed" for a past date (up to 7 days in the past) to correct missed entries.

6. Nonfunctional Requirements

6.1. Performance

NF-1.1: The website shall load the main dashboard page in less than 3 seconds for a user with a standard broadband connection.

NF-1.2: Marking a habit as complete (clicking the checkbox) shall register and update the display in less than 1 second.

NF-1.3: The website shall support at least100 concurrent users without significant degradation in performance.

6.2. Security

NF-2.1: All communication between the user's browser and the web server shall be encrypted using HTTPS.

NF-2.2: User passwords shall be stored securely using a strong hashing algorithm (e.g., bcrypt).

NF-2.3: Session cookies shall be secure and HTTP-only to prevent unauthorized access.

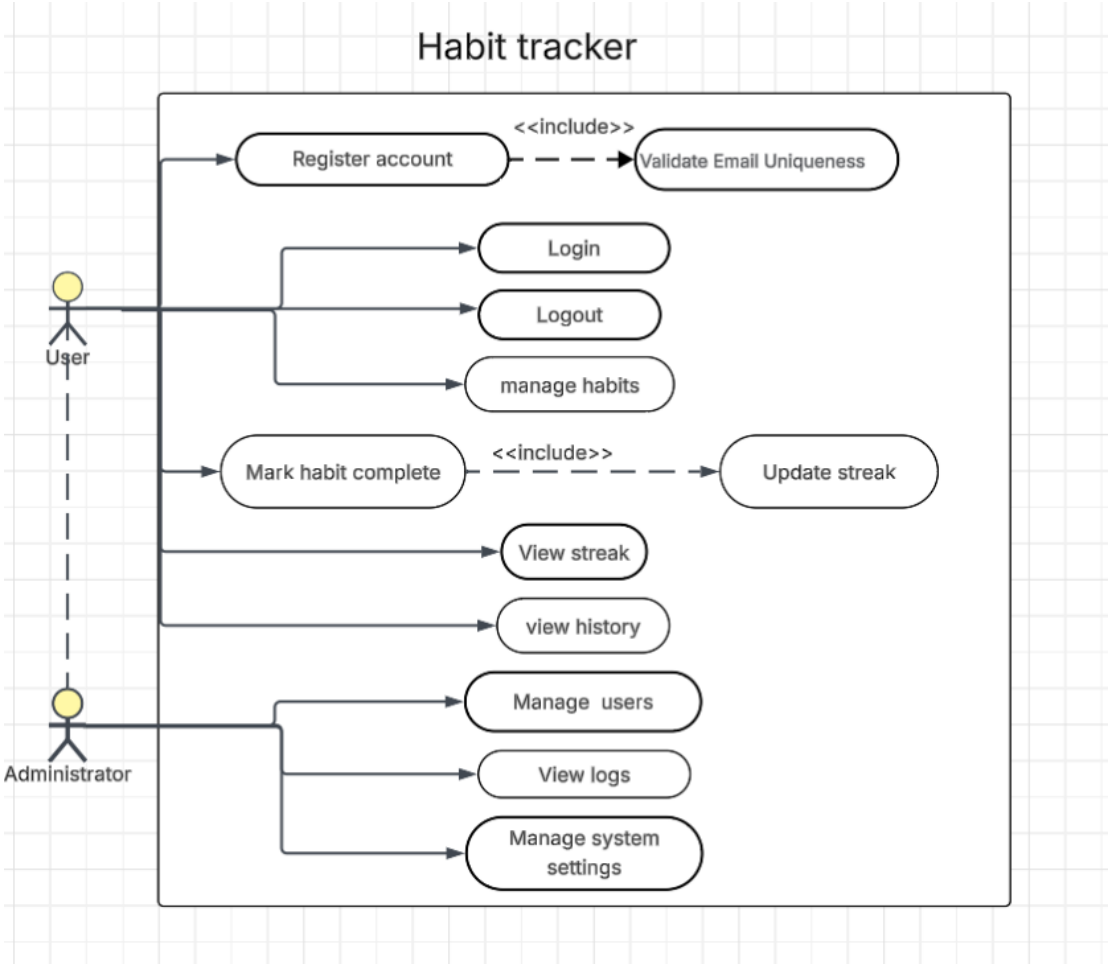

Figure 1. Use Case Diagram for the Habit Tracker System

The Use case diagram defines the functional boundaries of the DailyGlow system, identifying key participants and their goals. It illustrates two primary roles: the end user, who interacts with the core habit tracking features, and the administrator, responsible for system maintenance and user monitoring. Key interactions, such as account registration and habit management, are modeled to ensure the needs of all stakeholders within the application are considered.

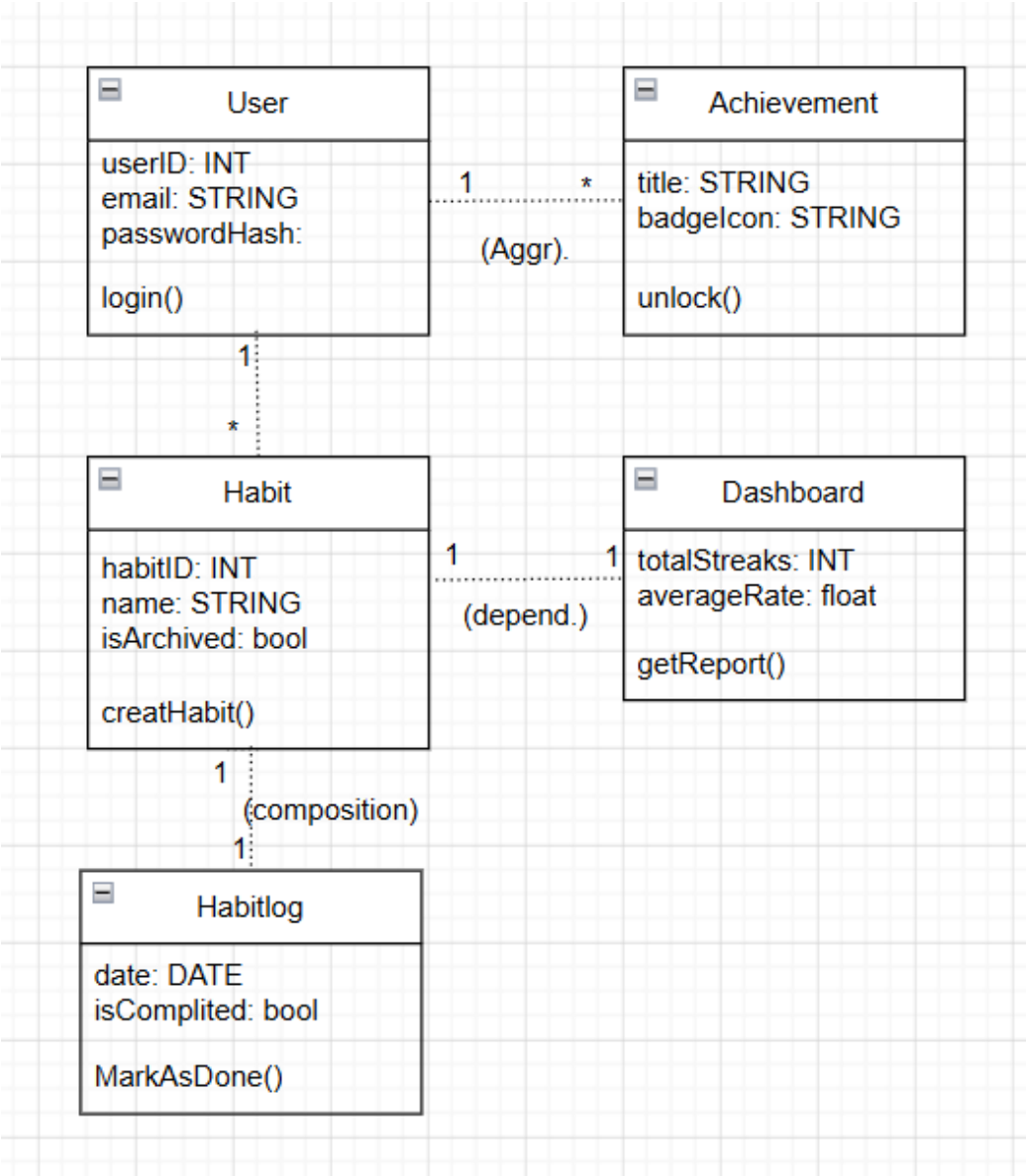

Figure 2. Class Diagram of the Application Architecture

To define the static structure of the database and backend logic, a UML Class Diagram was developed. This model outlines the essential entities, including User, Habit, and HabitLog, detailing their respective attributes and methods. The diagram captures critical relationships, such as the composition between habits and their logs, ensuring data consistency and providing a clear blueprint for the FastAPI implementation.

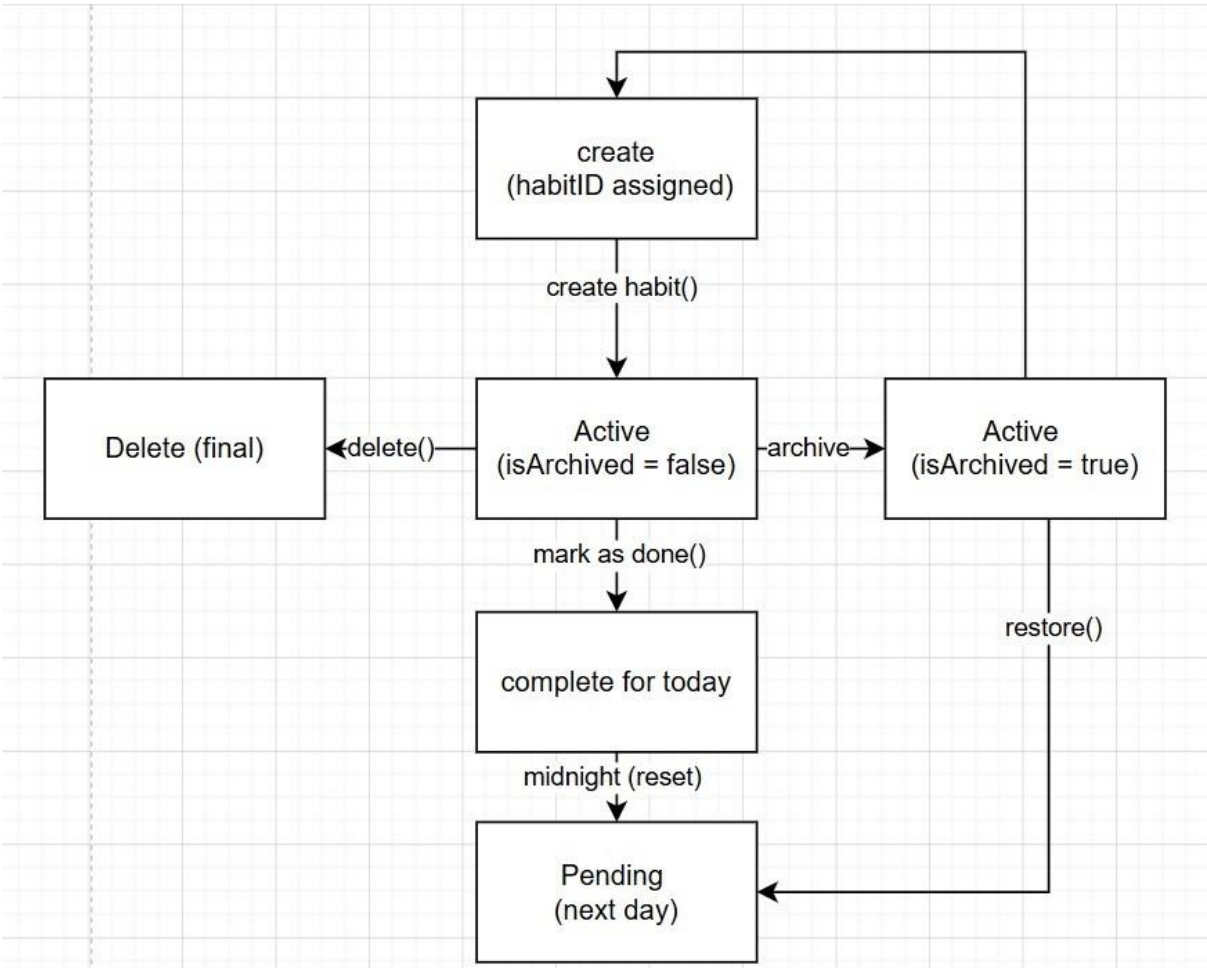

Figure 3. State Diagram

The State Machine Diagram illustrates the various stages in the lifecycle of a habit object within the system. It traces the transition from the Initial Creation (where a HabitID is assigned) to an Active state. The model highlights core behavioral logic: a habit can be marked as 'Complete for Today', after which it automatically resets to 'Pending' at midnight. Additionally, the diagram defines the administrative states, such as Archived and Deleted, ensuring that the system correctly manages data visibility and persistence according to the user's actions.

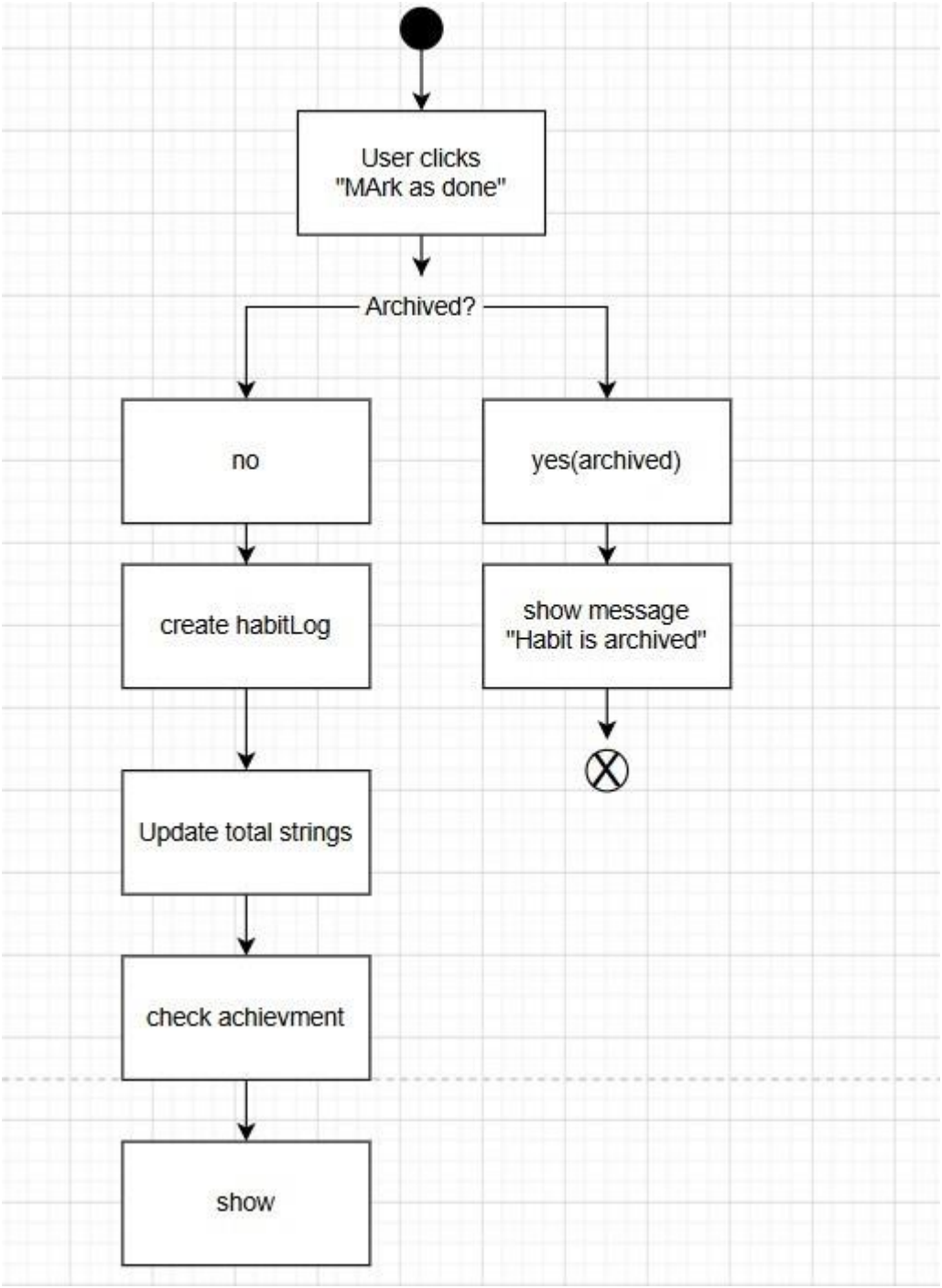

Figure 4. Activity Diagram

The Activity diagram details the procedural logic triggered when a user interacts with the "Mark as Completed" feature. It serves as a visual flowchart of the system's decision-making process. The process begins with a check to determine whether the habit is archived. If so, the system performs a series of sequential actions: creating a new entry in the habit log, recalculating the total number of streaks, and checking for newly unlocked achievements. This model ensures that all necessary background updates occur before the final status is displayed on the user's dashboard.

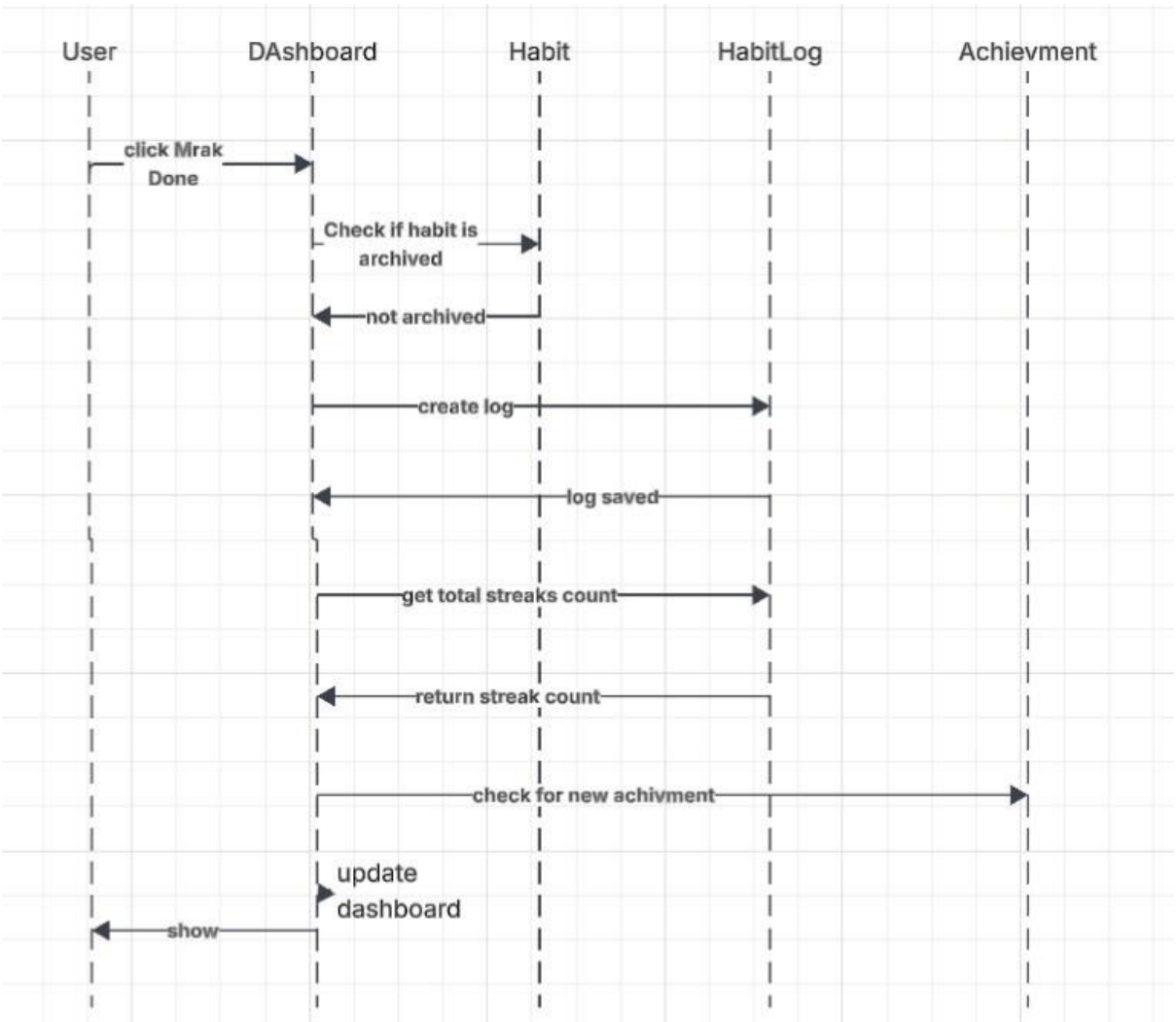

Figure 5. Sequence Diagram

The Sequence diagram provides a dynamic representation of the habit completion process, focusing on the chronological exchange of messages between system components. It visualizes the communication path from user interaction with the dashboard to the backend entities (Habit, Habit Log, and Achievement). The diagram confirms how the system maintains data integrity by checking the habit status, storing the log, and asynchronously receiving updates on the number of completed habits. This interaction model is crucial for ensuring a responsive user interface and accurate data synchronization in real time.